# Module II: System modeling, Software Design and Planning

## 1.1 Introduction to system modeling

**System modeling** is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system. System modeling has now come to mean representing a system using some kind of graphical notation, which is now almost always based on notations in the Unified Modeling Language (UML). System modelling helps the analyst to understand the functionality of the system and models are used to communicate with customers.

Models are used during the requirements engineering process to help derive the requirements for a system, during the design process to describe the system to engineers implementing the system and after implementation to document the system's structure and operation. You may develop models of both the existing system and the system to be developed:

1. Models of the existing system are used during requirements engineering. They help clarify what the existing system does and can be used as a basis for discussing its strengths and weaknesses. These then lead to requirements for the new system.
2. Models of the new system are used during requirements engineering to help explain the proposed requirements to other system stakeholders. Engineers use these models to discuss design proposals and to document the system for implementation. In a model-driven engineering process, it is possible to generate a complete or partial system implementation from the system model.

We may develop different models to represent the system from different perspectives. For example:

1. An external perspective, where you model the context or environment of the system.
2. An interaction perspective where you model the interactions between a system and its environment or between the components of a system.
3. A structural perspective, where you model the organization of a system or the structure of the data that is processed by the system.
4. A behavioral perspective, where you model the dynamic behavior of the system and how it responds to events.

Most users of the UML thought that five diagram types could represent the essentials of a system:

1. Activity diagrams, which show the activities involved in a process or in data processing.

2. Use case diagrams, which show the interactions between a system and its environment.
3. Sequence diagrams, which show interactions between actors and the system and between system components.
4. Class diagrams, which show the object classes in the system and the associations between these classes.
5. State diagrams, which show how the system reacts to internal and external events.

## 1.2 Context models

Context models are used to illustrate the operational context of a system - they show what lies outside the system boundaries. Social and organizational concerns may affect the decision on where to position system boundaries. Architectural models show the system and its relationship with other systems.

System boundaries are established to define what is inside and what is outside the system. They show other systems that are used or depend on the system being developed. The position of the system boundary has a profound effect on the system requirements. Defining a system boundary is a political judgment. There may be pressures to develop system boundaries that increase / decrease the influence or workload of different parts of an organization.

Figure 2.1 is a simple context model that shows the patient information system and the other systems in its environment. From Figure 2.1, you can see that the **MHC-PMS** is connected to an appointments system and a more general patient record system with which it shares data. The system is also connected to systems for management reporting and hospital bed allocation and a statistics system that collects information for research. Finally, it makes use of a prescription system to generate prescriptions for patients' medication.

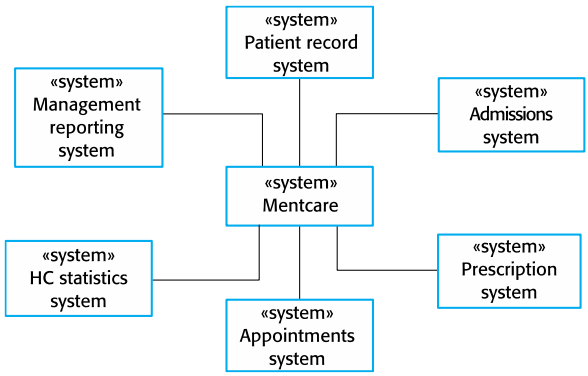



**Figure 2.1 The context of the MHC-PMS**

Context models normally show that the environment includes several other automated systems. However, they do not show the types of relationships between the systems in the environment and the system that is being specified. External systems might produce data

for or consume data from the system. They might share data with the system, or they might be connected directly, through a network or not connected at all. They might be physically co-located or located in separate buildings. All of these relations may affect the requirements and design of the system being defined and must be taken into account.

Therefore, simple context models are used along with other models, such as **business process models.** These describe human and automated processes in which particular software systems are used.

Figure 2.2 is a model of an important system process that shows the processes in which the MHC-PMS is used. Sometimes, patients who are suffering from mental health problems may be a danger to others or to themselves. They may therefore have to be detained against their will in a hospital so that treatment can be administered. Such detention is subject to strict legal safeguards—for example, the decision to detain a patient must be regularly reviewed so that people are not held indefinitely without good reason. One of the functions of the MHC-PMS is to ensure that such safeguards are implemented.

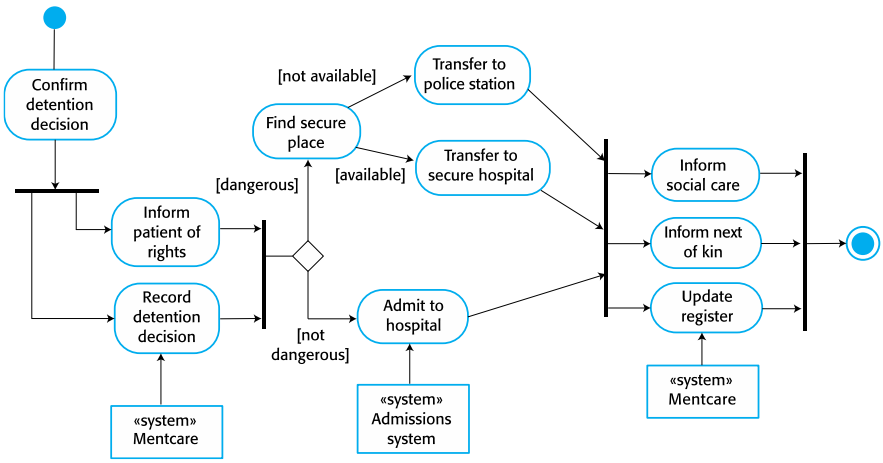



**Figure 2.2: Process model of involuntary detention**

Figure 2.2 is a **UML activity diagram**. Activity diagrams are intended to show the activities that make up a system process and the flow of control from one activity to another. The start of a process is indicated by a filled circle; the end by a filled circle inside another circle. Rectangles with round corners represent activities, that is, the specific sub-processes that must be carried out. You may include objects in activity charts.

In Figure 2.2, I have shown the systems that are used to support different processes. I have indicated that these are separate systems using the UML stereotype feature. In a UML activity diagram, arrows represent the flow of work from one activity to another. A solid bar is used to indicate activity coordination. When the flow from more than one activity leads to a solid bar then all of these activities must be complete before progress is possible. When the flow from a solid bar leads to a number of activities, these may be executed in parallel. Therefore, in Figure 5.2, the activities to inform social care and the patient's next of kin, and

to update the detention register may be concurrent. Arrows may be annotated with guards that indicate the condition when that flow is taken.

In Figure 2.2, you can see guards showing the flows for patients who are dangerous and not dangerous to society. Patients who are dangerous to society must be detained in a secure facility. However, patients who are suicidal and so are a danger to themselves may be detained in an appropriate ward in a hospital.

## 1.3 Interaction models

All systems involve interaction of some kind. This can be user interaction, which involves user inputs and outputs, interaction between the system being developed and other systems or interaction between the components of the system. Modeling user interaction is important as it helps to identify user requirements. Modeling system to system interaction highlights the communication problems that may arise. Modeling component interaction helps us understand if a proposed system structure is likely to deliver the required system performance and dependability.

### 1.3.1 Use case modeling

Use case modeling was originally developed by Jacobson et al. (1993) in the 1990s and was incorporated into the first release of the UML. Each use case represents a discrete task that involves external interaction with a system. In its simplest form, a use case is shown as an ellipse with the actors involved in the use case represented as stick figures.

Figure 2.3 shows a use case from the MHC-PMS that represents the task of uploading data from the MHC-PMS to a more general patient record system. This more general system maintains summary data about a patient rather than the data about each consultation, which is recorded in the MHC-PMS.

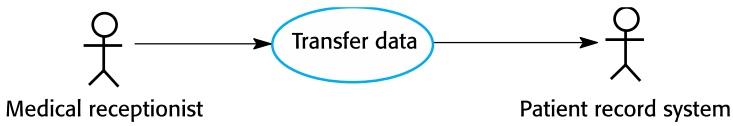



**Figure 2.3 Transfer data use case**

Use case diagrams give a fairly simple overview of an interaction so you have to provide more detail to understand what is involved. This detail can either be a simple textual description, a structured description in a table, or a sequence diagram as discussed below. Figure 2.4 shows a tabular description of the 'Transfer data' use case.

| MHC-PMS: Transfer data | |
|---|---|
| Actors | Medical receptionist, patient records system (PRS) |
| Description | A receptionist may transfer data from the MHC-PMS to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment. |
| Data | Patient's personal information, treatment summary |
| Stimulus | User command issued by medical receptionist |
| Response | Confirmation that PRS has been updated |
| Comments | The receptionist must have appropriate security permissions to access the patient information and the PRS. |

**Figure 2.4 Tabular description of the 'Transfer data' use case**

### 1.3.2 Sequence diagrams

Sequence diagrams in the UML are primarily used to model the interactions between the actors and the objects in a system and the interactions between the objects themselves. The UML has a rich syntax for sequence diagrams, which allows many different kinds of interaction to be modeled.

As the name implies, a sequence diagram shows the sequence of interactions that take place during a particular use case or use case instance. Figure 2.5 is an example of a sequence diagram that illustrates the basics of the notation. This diagram models the interactions involved in the View patient information use case, where a medical receptionist can see some patient information.

The objects and actors involved are listed along the top of the diagram, with a dotted line drawn vertically from these. Interactions between objects are indicated by annotated arrows. The rectangle on the dotted lines indicates the lifeline of the object concerned (i.e., the time that object instance is involved in the computation). You read the sequence of interactions from top to bottom. The annotations on the arrows indicate the calls to the objects, their parameters, and the return values. A box named alt is used with the conditions indicated in square brackets.

You can read Figure 2.5 as follows:

1. The medical receptionist triggers the ViewInfo method in an instance P of the PatientInfo object class, supplying the patient's identifier, PID. P is a user interface object, which is displayed as a form showing patient information.
2. The instance P calls the database to return the information required, supplying the receptionist's identifier to allow security checking.
3. The database checks with an authorization system that the user is authorized for this action.
4. If authorized, the patient information is returned and a form on the user's screen is filled in. If authorization fails, then an error message is returned.

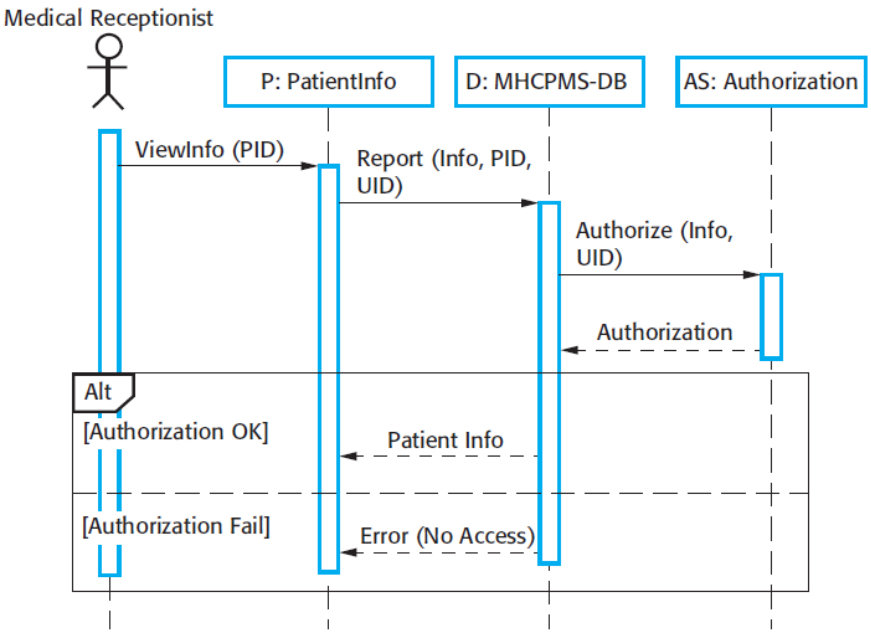



**Figure 2.5: Sequence diagram for View patient information**

Figure 2.6 is a second example of a sequence diagram from the same system that illustrates two additional features. These are the direct communication between the actors in the system and the creation of objects as part of a sequence of operations. In this example, an object of type Summary is created to hold the summary data that is to be uploaded to the PRS (patient record system). You can read this diagram as follows:

1. The receptionist logs on to the PRS.
2. There are two options available. These allow the direct transfer of updated patient information to the PRS and the transfer of summary health data from the MHC-PMS to the PRS.
3. In each case, the receptionist's permissions are checked using the authorization system.
4. Personal information may be transferred directly from the user interface object to the PRS.
5. On completion of the transfer, the PRS issues a status message and the user logs off.

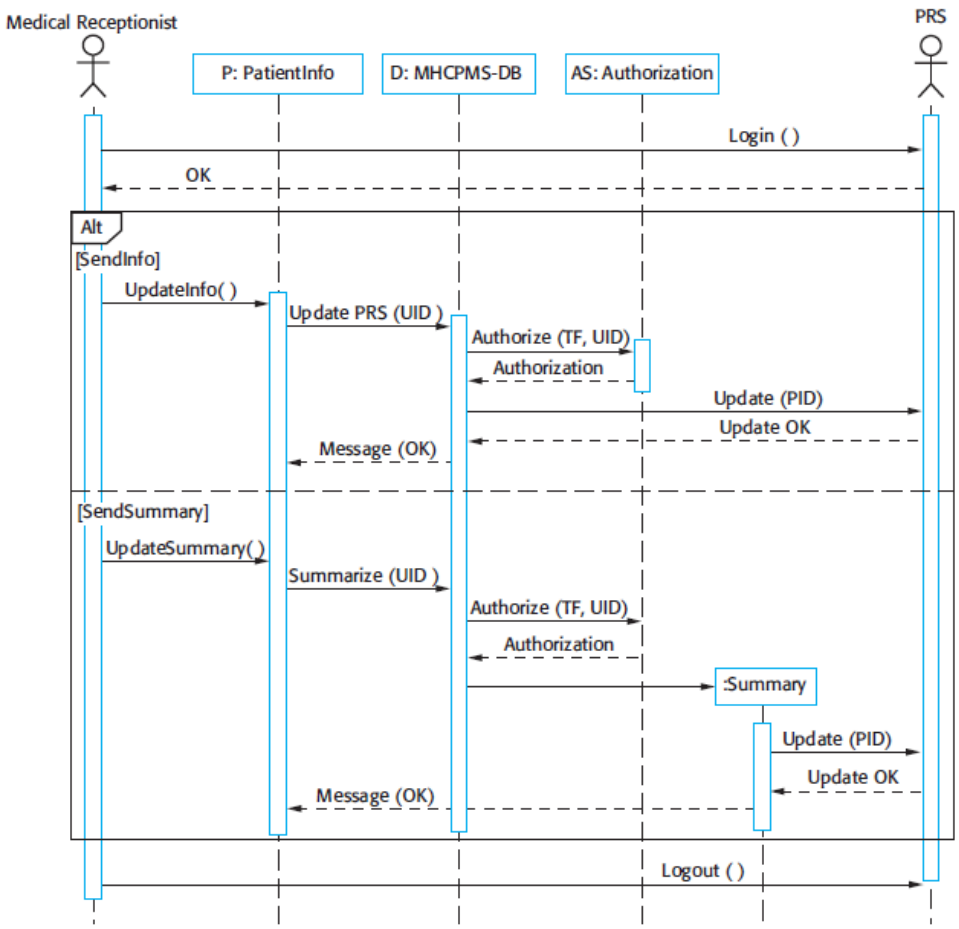



Figure 2.6: Sequence diagram for transfer data

## 1.4 Structural models

Structural models of software display the organization of a system in terms of the components that make up that system and their relationships. Structural models may be static models, which show the structure of the system design or dynamic models, which show the organization of the system when it is executing. These are not the same things—the dynamic organization of a system as a set of interacting threads may be very different from a static model of the system components.

### 1.4.1 Class diagrams

Class diagrams are used when developing an object-oriented system model to show the classes in a system and the associations between these classes. Loosely, an object class can be thought of as a general definition of one kind of system object. An association is a link between classes that indicates that there is a relationship between these classes. Consequently, each class may have to have some knowledge of its associated class.

When you are developing models during the early stages of the software engineering process, objects represent something in the real world, such as a patient, a prescription, a

doctor, etc. As an implementation is developed, you usually need to define additional implementation objects that are used to provide the required system functionality.

Class diagrams in the UML can be expressed at different levels of detail. The simplest way of writing these is to write the class name in a box. You can also simply note the existence of an association by drawing a line between classes. For example, Figure 2.7 is a simple class diagram showing two classes: Patient and Patient Record with an association between them.

In Figure 2.7, I illustrate a further feature of class diagrams—the ability to show how many objects are involved in the association. In this example, each end of the association is annotated with a 1, meaning that there is a 1:1 relationship between objects of these classes. That is, each patient has exactly one record and each record maintains information about exactly one patient.

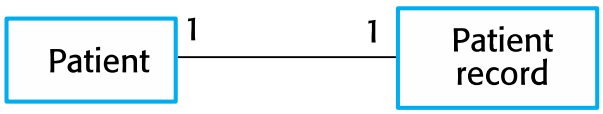



**Figure 2.7: UML classes and association**

Figure 2.8 develops this type of class diagram to show that objects of class Patient are also involved in relationships with a number of other classes.

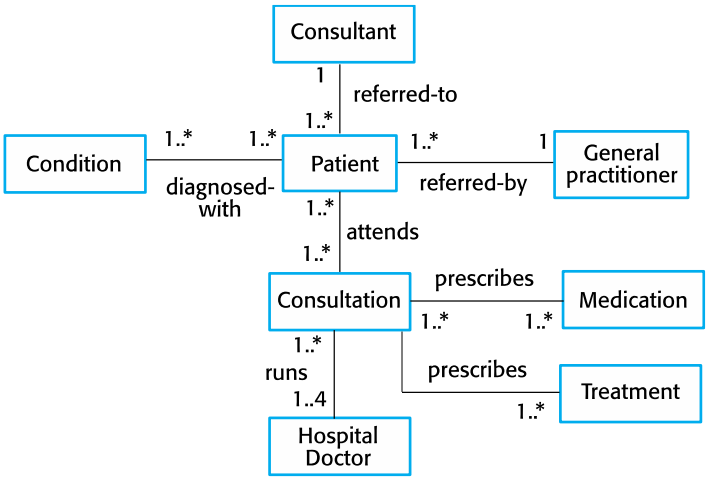



**Figure 2.8: Classes and associations in the MHC-PMS**

To define classes in more detail, you add information about their attributes (the characteristics of an object) and operations (the things that you can request from an object). For example, a Patient object will have the attribute Address and you may include an operation called ChangeAddress, which is called when a patient indicates that they have moved from one address to another. In the UML, you show attributes and operations by extending the simple rectangle that represents a class. This is illustrated in Figure 2.9 where:

1. The name of the object class is in the top section.

2. The class attributes are in the middle section. This must include the attribute names and, optionally, their types.
3. The operations (called methods in Java and other OO programming languages) associated with the object class are in the lower section of the rectangle.

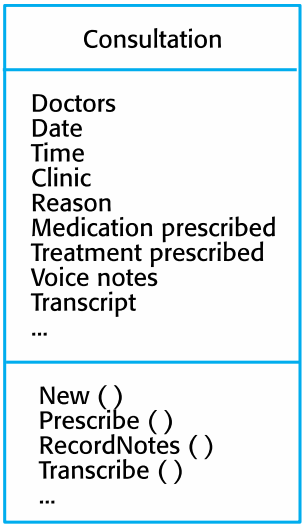



**Figure 2.9: The consultation class**

### 1.4.2 Generalization

Generalization is an everyday technique that we use to manage complexity. Rather than learn the detailed characteristics of every entity that we experience, we place these entities in more general classes (animals, cars, houses, etc.) and learn the characteristics of these classes. This allows us to infer that different members of these classes have some common characteristics e.g. squirrels and rats are rodents.

In modeling systems, it is often useful to examine the classes in a system to see if there is scope for generalization. If changes are proposed, then you do not have to look at all classes in the system to see if they are affected by the change. In object-oriented languages, such as Java, generalization is implemented using the class inheritance mechanisms built into the language.

The UML has a specific type of association to denote generalization, as illustrated in Figure 2.10. The generalization is shown as an arrowhead pointing up to the more general class. This shows that general practitioners and hospital doctors can be generalized as doctors and that there are three types of Hospital Doctor—those that have just graduated from medical school and have to be supervised (Trainee Doctor); those that can work unsupervised as part of a consultant's team (Registered Doctor); and consultants, who are senior doctors with full decision-making responsibilities.

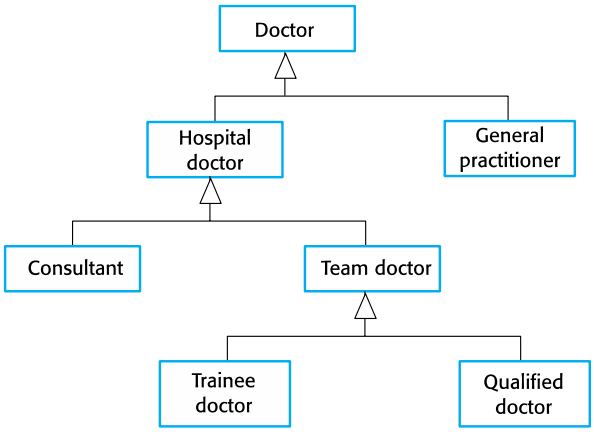



**Figure 2.10: A generalization hierarchy**

In a generalization, the attributes and operations associated with higher-level classes are also associated with the lower-level classes. In essence, the lower-level classes are subclasses inherit the attributes and operations from their super classes. These lower-level classes then add more specific attributes and operations. For example, all doctors have a name and phone number; all hospital doctors have a staff number and a department but general practitioners don't have these attributes as they work independently. They do however, have a practice name and address. This is illustrated in Figure 2.11, which shows part of the generalization hierarchy. The operations associated with the class Doctor are intended to register and de-register that doctor with the MHC-PMS.

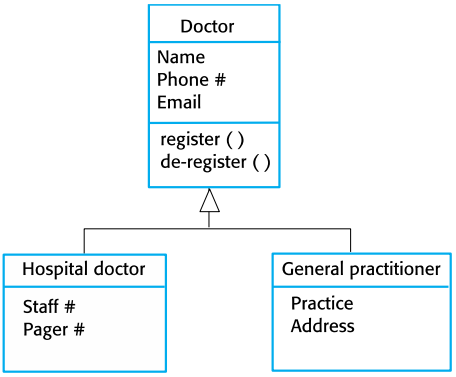



**Figure 2.11: A generalization hierarchy with added detail**

### 1.4.3 Aggregation

An aggregation model shows how classes that are collections are composed of other classes. Aggregation models are similar to the part-of relationship in semantic data models.

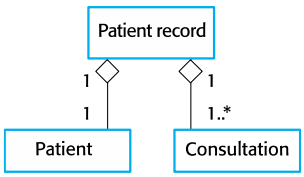



**Figure 2.12: The aggregation association**

## 1.5 Behavioral models

Behavioral models are models of the dynamic behavior of the system as it is executing. They show what happens or what is supposed to happen when a system responds to a stimulus from its environment. You can think of these stimuli as being of two types:

1. Data Some data arrives that has to be processed by the system.
2. Events Some event happens that triggers system processing. Events may have associated data but this is not always the case.

### 1.5.1 Data-driven modeling

Data-driven models show the sequence of actions involved in processing input data and generating an associated output. They are particularly useful during the analysis of requirements as they can be used to show end-to-end processing in a system. That is, they show the entire sequence of actions that take place from an input being processed to the corresponding output, which is the system's response. Data-flow models are useful because tracking and documenting how the data associated with a particular process moves through the system helps analysts and designers understand what is going on. Data-flow diagrams are simple and intuitive and it is usually possible to explain them to potential system users who can then participate in validating the model.

The UML does not support data-flow diagrams as they were originally proposed and used for modeling data processing. The reason for this is that DFDs focus on system functions and do not recognize system objects. However, because data-driven systems are so common in business, UML 2.0 introduced activity diagrams, which are similar to data-flow diagrams. For example, Figure 2.13 shows the chain of processing involved in the insulin pump software. In this diagram, you can see the processing steps (represented as activities) and the data flowing between these steps (represented as objects).

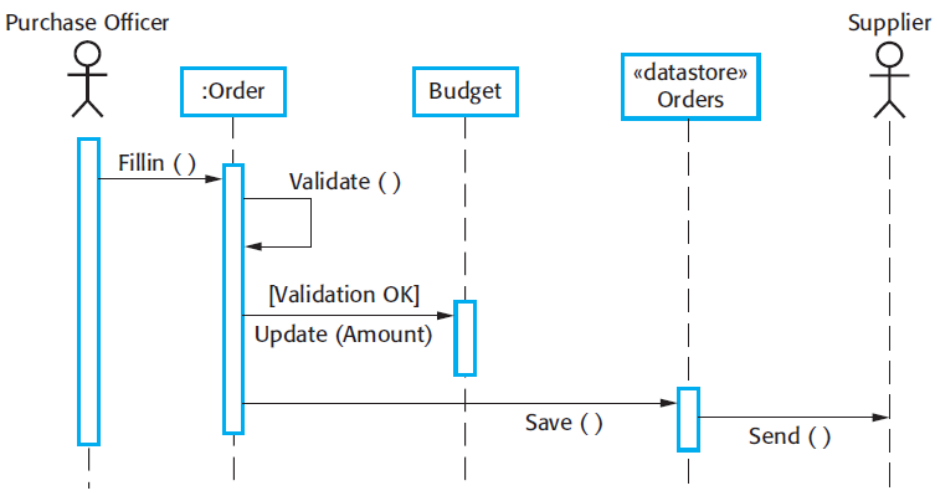



**Figure 2.13: An activity model of the insulin pump's operation.**

An alternative way of showing the sequence of processing in a system is to use UML sequence diagrams. You have seen how these can be used to model interaction but, if you draw these so that messages are only sent from left to right, then they show the sequential data processing in the system. Figure 2.14 illustrates this, using a sequence model of the processing of an order and sending it to a supplier.

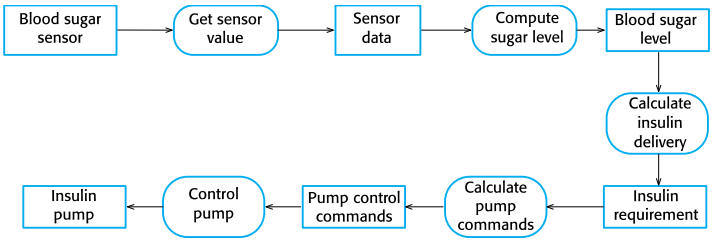



**Figure 2.14: Order processing**

## 1.5.2 Event-driven modeling

Event-driven modeling shows how a system responds to external and internal events. It is based on the assumption that a system has a finite number of states and that events (stimuli) may cause a transition from one state to another. For example, a system controlling a valve may move from a state 'Valve open' to a state 'Valve closed' when an operator command (the stimulus) is received. This view of a system is particularly appropriate for real-time systems.

The UML supports event-based modeling using state diagrams, which were based on State charts. State diagrams show system states and events that cause transitions from one state to another. They do not show the flow of data within the system but may include additional information on the computations carried out in each state. In UML state diagrams, rounded rectangles represent system states. They may include a brief description (following 'do') of the actions taken in that state. The labeled arrows represent stimuli that force a transition from one state to another. You can indicate start and end states using filled circles, as in activity diagrams. Figure 2.15 illustrates State diagram of a microwave oven.

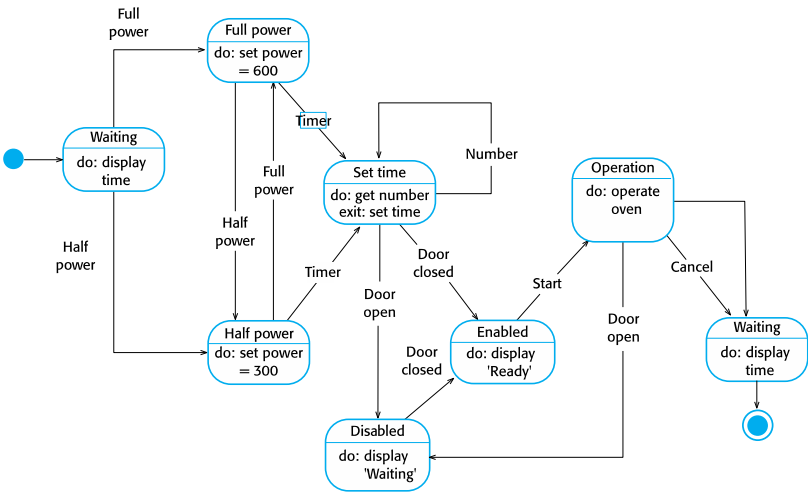



**Figure 2.15: State diagram of a microwave oven.**

Figure 2.16, which shows a tabular description of each state and how the stimuli that force state transitions are generated.

| State | Description |
|---|---|
| Waiting | The oven is waiting for input. The display shows the current time. |
| Half power | The oven power is set to 300 watts. The display shows 'Half power'. |
| Full power | The oven power is set to 600 watts. The display shows 'Full power'. |
| Set time | The cooking time is set to the user's input value. The display shows the cooking time selected and is updated as the time is set. |
| Disabled | Oven operation is disabled for safety. Interior oven light is on. Display shows 'Not ready'. |
| Enabled | Oven operation is enabled. Interior oven light is off. Display shows 'Ready to cook'. |
| Operation | Oven in operation. Interior oven light is on. Display shows the timer countdown. On completion of cooking, the buzzer is sounded for five seconds. Oven light is on. Display shows 'Cooking complete' while buzzer is sounding. |
| **Stimulus** | **Description** |
| Half power | The user has pressed the half-power button. |
| Full power | The user has pressed the full-power button. |
| Timer | The user has pressed one of the timer buttons. |
| Number | The user has pressed a numeric key. |
| Door open | The oven door switch is not closed. |
| Door closed | The oven door switch is closed. |
| Start | The user has pressed the Start button. |
| Cancel | The user has pressed the Cancel button. |

**Figure 2.16: States and stimuli for the microwave oven**

The problem with state-based modeling is that the number of possible states increases rapidly. For large system models, therefore, you need to hide detail in the models. One way to do this is by using the notion of a superstate that encapsulates a number of separate states. This superstate looks like a single state on a high-level model but is then expanded to show more detail on a separate diagram. This concept is illustrated in Figure 2.17. The Operation state includes a number of sub-states. It shows that operation starts with a status check and that if any problems are discovered an alarm is indicated and operation is disabled. Cooking involves running the microwave generator for the specified time; on completion, a buzzer is sounded. If the door is opened during operation, the system moves to the disabled state.

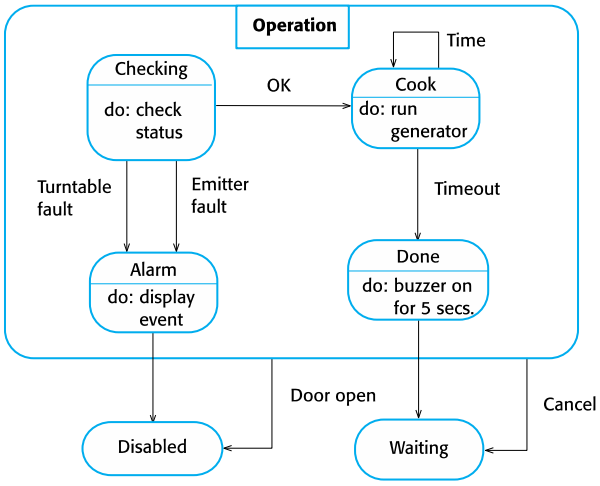



**Figure 2.17: Microwave oven operation**

## 1.6 Model-driven engineering

Model-driven engineering (MDE) is an approach to software development where models rather than programs are the principal outputs of the development process. The programs that execute on a hardware/software platform are then generated automatically from the models.

Proponents of MDE argue that this raises the level of abstraction in software engineering so that engineers no longer have to be concerned with programming language details or the specifics of execution platforms. Model-driven engineering is still at an early stage of development, and it is unclear whether or not it will have a significant effect on software engineering practice.

**Pros**

- Allows systems to be considered at higher levels of abstraction
- Generating code automatically means that it is cheaper to adapt systems to new platforms.

**Cons**

- Models for abstraction and not necessarily right for implementation.
- Savings from generating code may be outweighed by the costs of developing translators for new platforms.

### 1.6.1 Model-driven architecture

Model-driven architecture (MDA) was the precursor of more general model-driven engineering. MDA is a model-focused approach to software design and implementation that uses a subset of UML models to describe a system. Models at different levels of abstraction

are created. From a high-level, platform independent model, it is possible, in principle, to generate a working program without manual intervention.

The MDA method recommends that three types of abstract system model should be produced:

1. A computation independent model (CIM) that models the important domain abstractions used in the system. CIMs are sometimes called domain models. You may develop several different CIMs, reflecting different views of the system. For example, there may be a security CIM in which you identify important security abstractions such as an asset and a role and a patient record CIM, in which you describe abstractions such as patients, consultations, etc.
2. A platform independent model (PIM) that models the operation of the system without reference to its implementation. The PIM is usually described using UML models that show the static system structure and how it responds to external and internal events.
3. Platform specific models (PSM) which are transformations of the platform independent model with a separate PSM for each application platform. In principle, there may be layers of PSM, with each layer adding some platform specific detail. So, the first-level PSM could be middleware-specific but database independent. When a specific database has been chosen, a database specific PSM can then be generated.

Transformations between these models may be defined and applied automatically by software tools. This is illustrated in Figure 2.18, which also shows a final level of automatic transformation. A transformation is applied to the PSM to generate executable code that runs on the designated software platform.

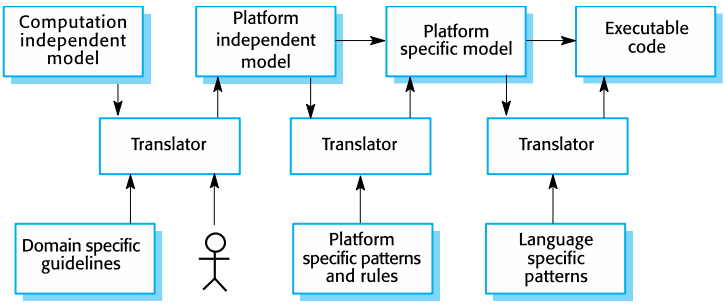



**Figure 2.18: MDA transformations.**

The translation of PIMs to PSMs is more mature and several commercial tools are available that provide translators from PIMs to common platforms such as Java and J2EE. These rely on an extensive library of platform-specific rules and patterns to convert the PIM to the PSM. There may be several PSMs for each PIM in the system. If a software system is intended to run on different platforms (e.g., J2EE and .NET), then it is only necessary to maintain the PIM. The PSMs for each platform are automatically generated. This is illustrated in Figure 2.19.

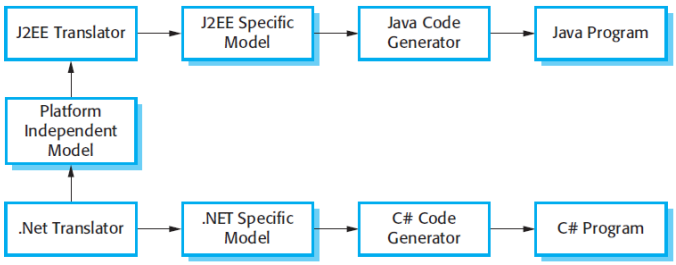



Figure 2.19: Multiple platform-specific models

## 1.7 Design and implementation

Software design and implementation is the stage in the software engineering process at which an executable software system is developed. Software design and implementation activities are invariably inter-leaved. Software design is a creative activity in which you identify software components and their relationships, based on a customer's requirements. Implementation is the process of realizing the design as a program.

### 1.7.1 Object-oriented design using the UML

Structured object-oriented design processes involve developing a number of different system models. They require a lot of effort for development and maintenance of these models and, for small systems, this may not be cost-effective. However, for large systems developed by different groups design models are an important communication mechanism.

There are a variety of different object-oriented design processes that depend on the organization using the process. Common activities in these processes include:

- Define the context and modes of use of the system;
- Design the system architecture;
- Identify the principal system objects;
- Develop design models;
- Specify object interfaces.

#### 1.7.1.1 System context and interactions

Understanding the relationships between the software that is being designed and its external environment is essential for deciding how to provide the required system functionality and how to structure the system to communicate with its environment.

Understanding of the context also lets you establish the boundaries of the system. Setting the system boundaries helps you decide what features are implemented in the system being designed and what features are in other associated systems.

System context models and interaction models present complementary views of the relationships between a system and its environment:

- A system context model is a structural model that demonstrates the other systems in the environment of the system being developed.
- An interaction model is a dynamic model that shows how the system interacts with its environment as it is used.

The context model of a system may be represented using associations. Associations simply show that there are some relationships between the entities involved in the association. The nature of the relationships is now specified. You may therefore document the environment of the system using a simple block diagram, showing the entities in the system and their associations. This is illustrated in Figure 2.20, which shows that the systems in the environment of each weather station are a weather information system, an onboard satellite system, and a control system. The cardinality information on the link shows that there is one control system but several weather stations, one satellite, and one general weather information system.

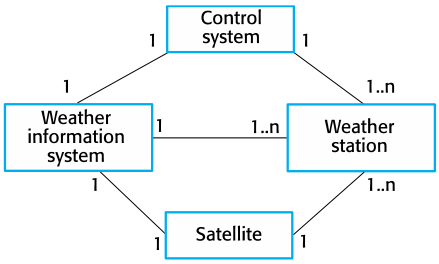



**Figure 2.20: System context for the weather station**

The use case model for the weather station is shown in Figure 2.21. This shows that the weather station interacts with the weather information system to report weather data and the status of the weather station hardware. Other interactions are with a control system that can issue specific weather station control commands.

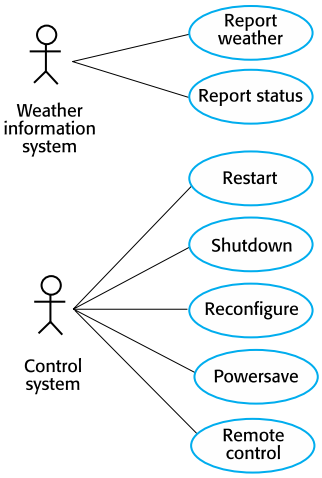



**Figure 2.21: Weather Control station use cases**

### 1.7.1.2 Architectural design

Once the interactions between the software system and the system's environment have been defined, you use this information as a basis for designing the system architecture. Of course, you need to combine this with your general knowledge of the principles of architectural design and with more detailed domain knowledge.

The high-level architectural design for the weather station software is shown in Figure 2.22. The weather station is composed of independent subsystems that communicate by broadcasting messages on a common infrastructure, shown as the Communication link in Figure 2.22. Each subsystem listens for messages on that infrastructure and picks up the messages that are intended for them.

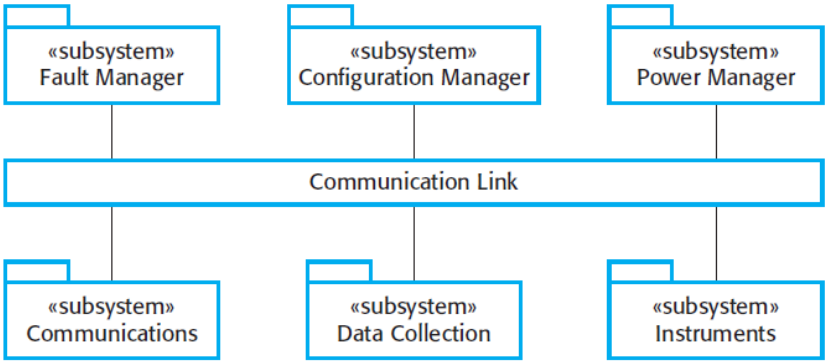



**Figure 2.22: High-level architecture of the weather station**

Figure 2.23 shows the architecture of the data collection subsystem, which is included in Figure 2.22. The Transmitter and Receiver objects are concerned with managing communications and the WeatherData object encapsulates the information that is collected from the instruments and transmitted to the weather information system.

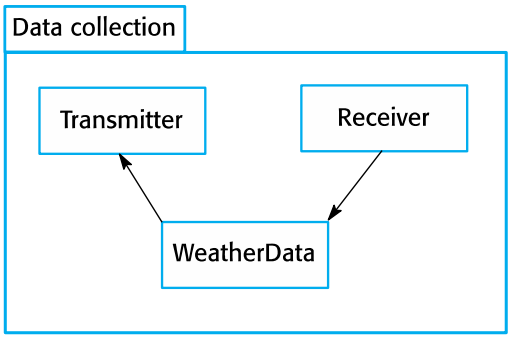



**Figure 2.23: Architecture WeatherData of data collection system**

### 1.7.1.3 Object class identification

Identifying object classes is often a difficult part of object-oriented design. There is no 'magic formula' for object identification. It relies on the skill, experience and domain knowledge of

system designers. Object identification is an iterative process. You are unlikely to get it right first time. There have been various proposals made about how to identify object classes in object-oriented systems:

- ✧ Use a **grammatical approach** based on a natural language description of the system.
- ✧ Base the identification on **tangible things** in the application domain such as aircraft, roles such as manager or doctor, events such as requests, interactions such as meetings, locations such as offices, organizational units such as companies, and so on
- ✧ Use a **behavioural approach** and identify objects based on what participates in what behaviour.
- ✧ Use a **scenario-based analysis**. The objects, attributes and methods in each scenario are identified.

In the wilderness weather station, object identification is based on the tangible hardware in the system. I have shown five object classes in Figure 2.24. The Ground thermometer, Anemometer, and Barometer objects are application domain objects, and the WeatherStation and WeatherData objects have been identified from the system description and the scenario (use case) description:

- **Ground thermometer, Anemometer, Barometer:** Application domain objects that are 'hardware' objects related to the instruments in the system.
- **Weather station:** The basic interface of the weather station to its environment. It therefore reflects the interactions identified in the use-case model.
- **Weather data:** Encapsulates the summarized data from the instruments.

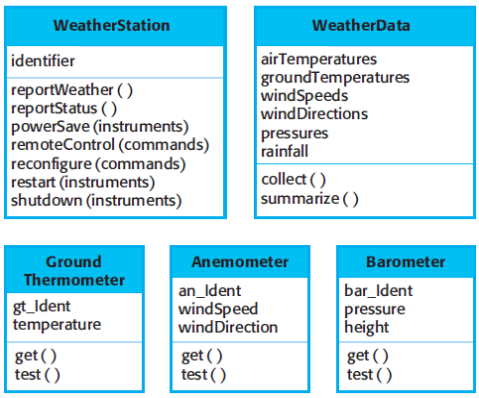



**Figure 2.24: Weather station objects**

### 1.7.1.4 Design models

Design models show the objects and object classes and relationships between these entities. There are two kinds of design model:

1. Structural models describe the static structure of the system in terms of object classes and relationships.

2. Dynamic models describe the dynamic interactions between objects.

**Examples of design models:**

- **Subsystem models** that show logical groupings of objects into coherent subsystems.
- **Sequence models** that show the sequence of object interactions.
- **State machine models** that show how individual objects change their state in response to events.
- **Other models** include use-case models, aggregation models, generalisation models, etc.

A subsystem model shows how the design is organized into logically related groups of objects. In the UML, these are shown using **packages** - an encapsulation construct. This is a logical model. The actual organization of objects in the system may be different.

Sequence models are dynamic models that describe, for each mode of interaction, the sequence of object interactions that take place. When documenting a design, you should produce a sequence model for each significant interaction. If you have developed a use case model, then there should be a sequence model for each use case that you have identified.

Figure 2.25 is an example of a sequence model, shown as a UML sequence diagram. This diagram shows the sequence of interactions that take place when an external system requests the summarized data from the weather station. You read sequence diagrams from top to bottom:

1. The SatComms object receives a request from the weather information system to collect a weather report from a weather station. It acknowledges receipt of this request. The stick arrowhead on the sent message indicates that the external system does not wait for a reply but can carry on with other processing.
2. SatComms sends a message to WeatherStation, via a satellite link, to create a summary of the collected weather data. Again, the stick arrowhead indicates that SatComms does not suspend itself waiting for a reply.
3. WeatherStation sends a message to a Commslink object to summarize the weather data. In this case, the squared-off style of arrowhead indicates that the instance of the WeatherStation object class waits for a reply.
4. Commslink calls the summarize method in the object WeatherData and waits for a reply.
5. The weather data summary is computed and returned to WeatherStation via the Commslink object.
6. WeatherStation then calls the SatComms object to transmit the summarized data to the weather information system, through the satellite communications system.

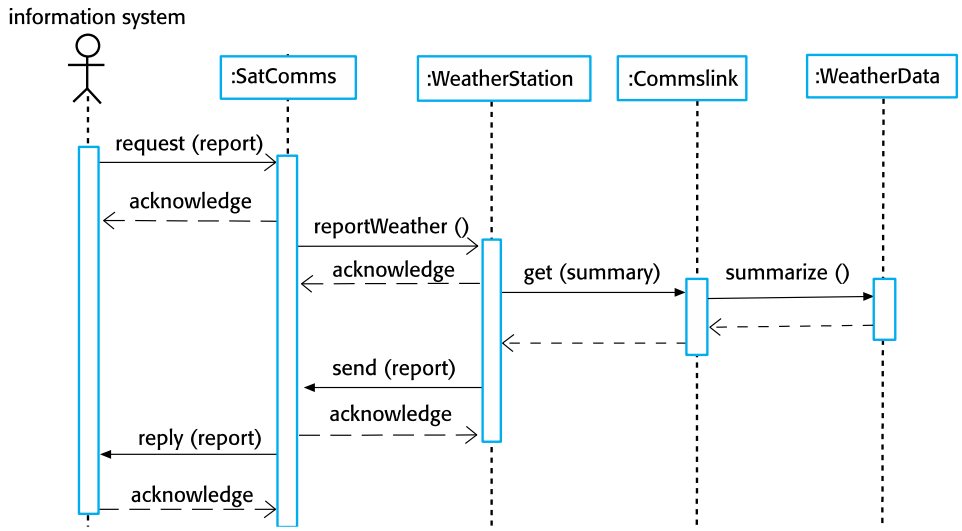



**Figure 2.25: Sequence diagram describing data collection**

**State diagrams** are used to show how objects respond to different service requests and the state transitions triggered by these requests. State diagrams are useful high-level models of a system or an object's run-time behavior. You don't usually need a state diagram for all of the objects in the system. Many of the objects in a system are relatively simple and a state model adds unnecessary detail to the design. Figure 2.26 is a state diagram for the weather station system that shows how it responds to requests for various services. You can read this diagram as follows:

1. If the system state is Shutdown then it can respond to a restart(), a reconfigure(), or a powerSave() message. The unlabeled arrow with the black blob indicates that the Shutdown state is the initial state. A restart() message causes a transition to normal operation. Both the powerSave() and reconfigure() messages cause a transition to a state in which the system reconfigures itself. The state diagram shows that reconfiguration is only allowed if the system has been shut down.
2. In the Running state, the system expects further messages. If a shutdown() message is received, the object returns to the shutdown state.
3. If a reportWeather() message is received, the system moves to the Summarizing state. When the summary is complete, the system moves to a Transmitting state where the information is transmitted to the remote system. It then returns to the Running state.
4. If a reportStatus() message is received, the system moves to the Testing state, then the Transmitting state, before returning to the Running state.
5. If a signal from the clock is received, the system moves to the Collecting state, where it collects data from the instruments. Each instrument is instructed in turn to collect its data from the associated sensors.

6. If a remoteControl() message is received, the system moves to a controlled state in which it responds to a different set of messages from the remote control room. These are not shown on this diagram.

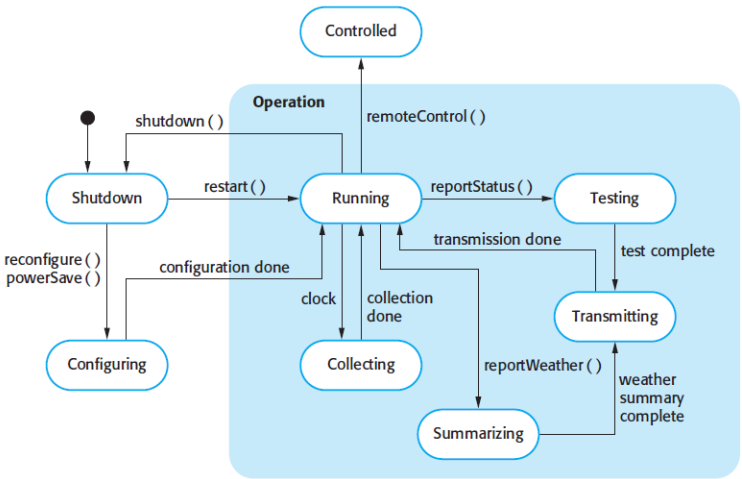



Figure 2.26: Weather station state diagram

### 1.7.1.5 Interface specification

Object interfaces have to be specified so that the objects and other components can be designed in parallel. Designers should avoid designing the interface representation but should hide this in the object itself. Objects may have several interfaces which are viewpoints on the methods provided. The UML uses class diagrams for interface specification but Java may also be used. Figure 2.27 shows two interfaces that may be defined for the weather station. The left-hand interface is a reporting interface that defines the operation names that are used to generate weather and status reports. These map directly to operations in the WeatherStation object. The remote control interface provides four operations, which map onto a single method in the WeatherStation object.

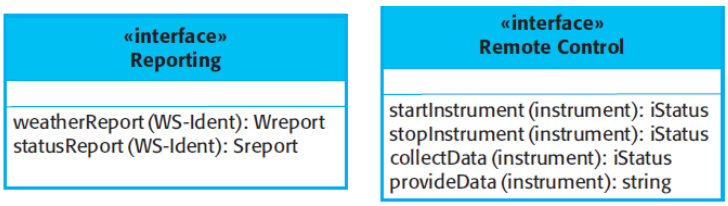



Figure 2.27: Weather station interfaces

## 1.7.2 Design patterns

Design patterns were derived from ideas put forward by Christopher Alexander, who suggested that there were certain common patterns of building design that were inherently pleasing and effective. The **pattern** is a description of the problem and the essence of its solution, so that the solution may be reused in different settings. The pattern is not a detailed

specification. Rather, you can think of it as a description of accumulated wisdom and experience, a well-tried solution to a common problem.

A quote from the Hillside Group web site (http://hillside.net), which is dedicated to maintaining information about patterns, encapsulates their role in reuse:

*Patterns and Pattern Languages are ways to describe best practices, good designs, and capture experience in a way that it is possible for others to reuse this experience.*

Patterns have made a huge impact on object-oriented software design. As well as being tested solutions to common problems, they have become a vocabulary for talking about a design. Design patterns are usually associated with object-oriented design. Published patterns often rely on object characteristics such as inheritance and polymorphism to provide generality.

**Pattern elements**

1. A name that is a meaningful reference to the pattern.
2. A description of the problem area that explains when the pattern may be applied.
3. A solution description of the parts of the design solution, their relationships, and their responsibilities. This is not a concrete design description. It is a template for a design solution that can be instantiated in different ways. This is often expressed graphically and shows the relationships between the objects and object classes in the solution.
4. A statement of the consequences—the results and trade-offs—of applying the pattern. This can help designers understand whether or not a pattern can be used in a particular situation.

**Example: The Observer pattern**

- Name
  - Observer.
- Description
  - Separates the display of object state from the object itself.
- Problem description
  - Used when multiple displays of state are needed.
- Solution description
  - See slide with UML description.
- Consequences
  - Optimisations to enhance display performance are impractical.

The Observer pattern can be used in situations where different presentations of an object's state are required. It separates the object that must be displayed from the different forms of presentation. This is illustrated in Figure 2.28, which shows two graphical

presentations of the same data set. Figure 2.29 is the representation in UML of the Observer pattern.

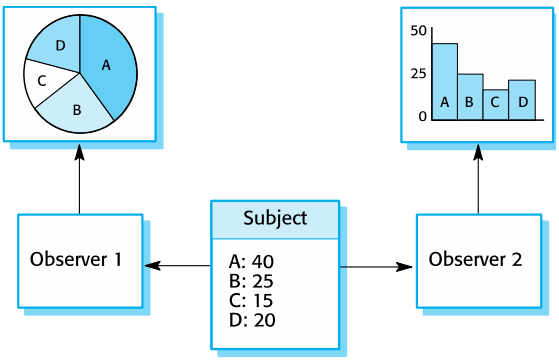



**Figure 2.28: Multiple displays**

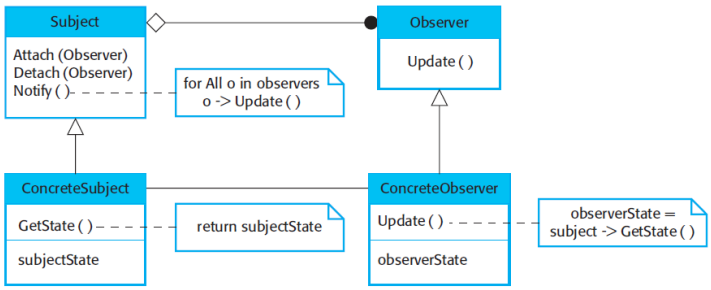



**Figure 2.29: A UML model of the Observer pattern**

To use patterns in your design, you need to recognize that any design problem you are facing may have an associated pattern that can be applied.

- Tell several objects that the state of some other object has changed (**Observer pattern**).
- Tidy up the interfaces to a number of related objects that have often been developed incrementally (**Façade pattern**).
- Provide a standard way of accessing the elements in a collection, irrespective of how that collection is implemented (**Iterator pattern**).
- Allow for the possibility of extending the functionality of an existing class at run-time (**Decorator pattern**).

### 1.7.3 Implementation issues

Focus here is not on programming, although this is obviously important, but on other implementation issues that are often not covered in programming texts:

- **Reuse:** Most modern software is constructed by reusing existing components or systems. When you are developing software, you should make as much use as possible of existing code.

- **Configuration management**: During the development process, you have to keep track of the many different versions of each software component in a configuration management system.
- **Host-target development Production:** Software does not usually execute on the same computer as the software development environment. Rather, you develop it on one computer (the host system) and execute it on a separate computer (the target system).

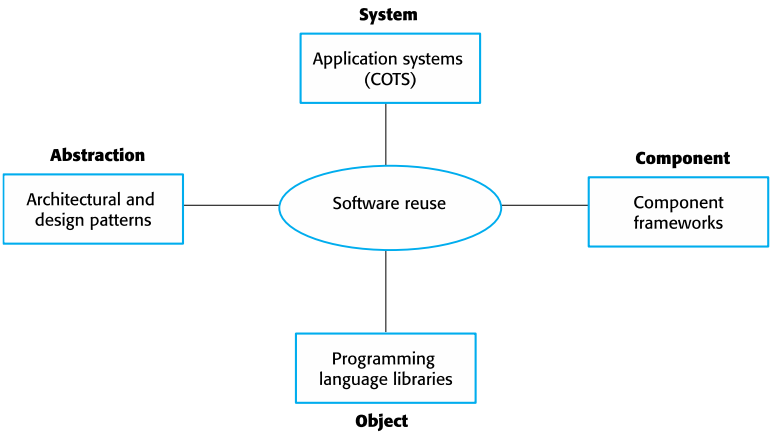
**Reuse**

- ✧ From the 1960s to the 1990s, newest software was developed from scratch, by writing all code in a high-level programming language.
  - The only significant reuse or software was the reuse of functions and objects in programming language libraries.
- ✧ Costs and schedule pressure mean that this approach became increasingly unviable, especially for commercial and Internet-based systems.
- ✧ An approach to development based around the reuse of existing software emerged and is now generally used for business and scientific software.

Software reuse is possible at a number of different levels:

1. **The abstraction level:** At this level, you don't reuse software directly but rather use knowledge of successful abstractions in the design of your software. Design patterns and architectural patterns are ways of representing abstract knowledge for reuse.
2. **The object level:** At this level, you directly reuse objects from a library rather than writing the code yourself. To implement this type of reuse, you have to find appropriate libraries and discover if the objects and methods offer the functionality that you need. For example, if you need to process mail messages in a Java program, you may use objects and methods from a JavaMail library.
3. **The component level:** Components are collections of objects and object classes that operate together to provide related functions and services. You often have to adapt and extend the component by adding some code of your own. An example of component-level reuse is where you build your user interface using a framework.
4. **The system level:** At this level, you reuse entire application systems. This usually involves some kind of configuration of these systems. This may be done by adding and modifying code or by using the system's own configuration interface. Most commercial systems are now built in this way where generic COTS (commercial off-the-shelf) systems are adapted and reused. Sometimes this approach may involve reusing several different systems and integrating these to create a new system.

The below figure shows software reuse.




By reusing existing software, you can develop new systems more quickly, with fewer development risks and also lower costs. As the reused software has been tested in other applications, it should be more reliable than new software. However, there are **costs associated with reuse:**

- The costs of the time spent in looking for software to reuse and assessing whether or not it meets your needs.
- Where applicable, the costs of buying the reusable software. For large off-the-shelf systems, these costs can be very high.
- The costs of adapting and configuring the reusable software components or systems to reflect the requirements of the system that you are developing.
- The costs of integrating reusable software elements with each other (if you are using software from different sources) and with the new code that you have developed.

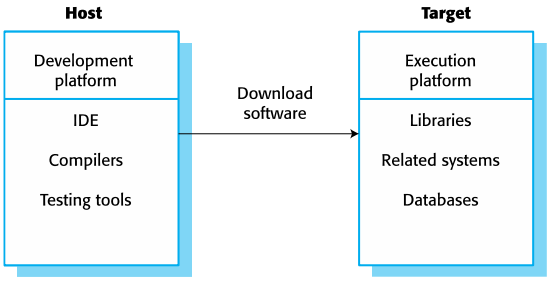
**Configuration management**

Configuration management is the name given to the general process of managing a changing software system. The aim of configuration management is to support the system integration process so that all developers can access the project code and documents in a controlled way, find out what changes have been made, and compile and link components to create a system. There are, therefore, three fundamental configuration management activities:

1. **Version management**: where support is provided to keep track of the different versions of software components. Version management systems include facilities to coordinate development by several programmers. They stop one developer overwriting code that has been submitted to the system by someone else.
2. **System integration**: where support is provided to help developers define what versions of components are used to create each version of a system. This description is then used to build a system automatically by compiling and linking the required components.
3. **Problem tracking**: where support is provided to allow users to report bugs and other problems, and to allow all developers to see who is working on these problems and when they are fixed.

**Host-target development**

- Most software is developed on one computer (the host), but runs on a separate machine (the target).
- More generally, we can talk about a development platform and an execution platform.
  - A platform is more than just hardware.
  - It includes the installed operating system plus other supporting software such as a database management system or, for development platforms, an interactive development environment.
- Development platform usually has different installed software than execution platform; these platforms may have different architectures. It is shown in below figure.




A software development platform should provide **a range of tools** to support software engineering processes. These may include:

1. An integrated compiler and syntax-directed editing system that allows you to create, edit, and compile code.
2. A language debugging system.
3. Graphical editing tools, such as tools to edit UML models.
4. Testing tools, such as JUnit (Massol, 2003) that can automatically run a set of tests on a new version of a program.
5. Project support tools that help you organize the code for different development projects.

Software development tools are often grouped to create an **integrated development environment (IDE)**. An IDE is a set of software tools that supports different aspects of software development, within some common framework and user interface. Generally, IDEs are created to support development in a specific programming language such as Java. The language IDE may be developed specially, or may be an instantiation of a general-purpose IDE, with specific language-support tools.

A general-purpose IDE is a framework for hosting software tools that provides data management facilities for the software being developed, and integration mechanisms, that

allow tools to work together. The best-known general-purpose IDE is the **Eclipse** environment. This environment is based on a plug-in architecture so that it can be specialized for different languages and application domains.

As part of the development process, you need to make decisions about how the developed software will be deployed on the target platform. Issues that you have to consider in making this decision are:

1. **The hardware and software requirements of a component**: If a component is designed for a specific hardware architecture, or relies on some other software system, it must obviously be deployed on a platform that provides the required hardware and software support.
2. **The availability requirements of the system:** High-availability systems may require components to be deployed on more than one platform. This means that, in the event of platform failure, an alternative implementation of the component is available.
3. **Component communications:** If there is a high level of communications traffic between components, it usually makes sense to deploy them on the same platform or on platforms that are physically close to one other. This reduces communications latency, the delay between the time a message is sent by one component and received by another.

### 1.7.4 Open source development

Open source development is an approach to software development in which the source code of a software system is published and volunteers are invited to participate in the development process. Its roots are in the Free Software Foundation (www.fsf.org), which advocates that source code should not be proprietary but rather should always be available for users to examine and modify as they wish. Open source software extended this idea by using the Internet to recruit a much larger population of volunteer developers. Many of them are also users of the code.

The best-known open source product is, of course, the **Linux** operating system which is widely used as a server system and, increasingly, as a desktop environment. Other important open source products are Java, the Apache web server and the mySQL database management system.

For a company involved in software development, there are two open source issues that have to be considered:

1. Should the product that is being developed make use of open source components?
2. Should an open source approach be used for the software's development?

### Open source licensing

Although a fundamental principle of open-source development is that source code should be freely available, this does not mean that anyone can do as they wish with that code. Legally, the developer of the code still owns the code. They can place restrictions on how it is used by including legally binding conditions in an open source software license. Some open source developers believe that if an open source component is used to develop a new system, then that system should also be open source. Others are willing to allow their code to be used without this restriction. The developed systems may be proprietary and sold as closed source systems.

Most open source licenses are derived from one of three general models:

1. The GNU General Public License (GPL). This is a so-called 'reciprocal' license that, simplistically, means that if you use open source software that is licensed under the GPL license, then you must make that software open source.
2. The GNU Lesser General Public License (LGPL). This is a variant of the GPL license where you can write components that link to open source code without having to publish the source of these components. However, if you change the licensed component, then you must publish this as open source.
3. The Berkley Standard Distribution (BSD) License. This is a non-reciprocal license, which means you are not obliged to republish any changes or modifications made to open source code. You can include the code in proprietary systems that are sold. If you use open source components, you must acknowledge the original creator of the code.

## End of Module 2